Instructor Handout 7: Operations on Processes

Lecture 7 — Creation, Termination, `fork/exec`

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Instructor Copy — Not for Student Distribution

Concepts That Click

7.1 Before You Teach This Lecture

- **Silberschatz, Galvin & Gagne, *Operating System Concepts*, 8e** — Ch. 3 §3.3, operations on processes (pp. 110–116): the process tree, the three parental decisions (resources, execution, address space), `fork`/`exec`/`wait`, the Windows `CreateProcess` contrast, termination reasons, and cascading termination. Primary anchor. *Page-verified* via the course index.
- **Stallings, *Operating Systems: Internals and Design Principles*, 6e** — Ch. 3 §3.4, process control (p. 135), for the mode-switch and PCB context around creation. *Page-verified.*
- **Bovet & Cesati, *Understanding the Linux Kernel*, 3e** — Ch. 3 (pp. 79–130) for `fork`, `clone`, `vfork`, copy-on-write, and destroying processes (the zombie state); Ch. 20 (pp. 808–834) for `execve`. Use it for concrete Linux detail only. *Page-verified.*
- **Bach, *The Design of the UNIX Operating System*** — chapter-level reference for the classical UNIX process view. *Not indexed; no page numbers.*
- **Run the programs before class.** Every count in this handout was verified by compiling and running it. Re-run the four you will demo (three forks; `(fork() && fork()) || fork()`; the `"hello "` buffer trap; `exit(7)`) so you can show live output. Run the buffer trap piped through `cat` — the duplicated `hello` is unmistakable.
- **Prerequisites.** Lecture 3 (`printf` buffers before `write`; the process-control category) and Lectures 5–6 (PCB, process states). Remind students of the stdio buffer before Section 7.3; the trap depends on it.

Teaching note: The thesis to keep on the board all hour: *`fork` copies, `exec` replaces, `exit` reports, `wait` collects.* Every section adds detail to one of those four verbs.

7.2 Opening: Who Actually Runs `ls`?

Type `ls` at a prompt. A listing appears and the prompt returns. The shell did not turn into `ls` — it is still there — and `ls` did not exist as a process a moment earlier. So a process was born, ran, and vanished, and the shell knew exactly when it finished. Every process except the first is created by another process: the shell creates a **child**, the child becomes `ls`, and the shell **waits**. `fork`, `exec`, and `wait` do the work, and `exit` ends the child. The lecture follows this one command from creation to termination, through four questions: how a process is created; what `fork` returns and how many processes we get; how a child becomes a different program; and how a process ends and who cleans up. Lecture 5 gave the PCB and Lecture 6 moved it between states; today we create and destroy it.

Board: the four verbs. *On the board, in order:* (i) write `fork`, `exec`, `exit`, `wait` in a column on the left; (ii) beside each, ask the class who calls it — shell or child — and write the answer only after hands go up (shell, child, child, shell); (iii) circle the two *child* calls. *Say:* "The shell does two things and the child does two things. Today is about why the work is divided exactly that way."

Board: what students already see. *On the board, in order:* (i) write `ps -f` and its typical columns `PID PPID CMD`; (ii) write one row for the shell and one row for `ps` itself; (iii) draw an arrow from the `ps` row's PPID to the shell's PID. *Say:* "Every row has a parent column. That column is today's lecture."

> **Teaching note:** Ask "Did the shell run `ls`?" before anything else. Most students say yes. The correction — the shell *made a copy of itself* that became `ls` — is the first surprise and sets up `fork`.

7.3 Process Creation

The process tree

Definition (Parent, child, process tree). *A **parent process** is a process that issues a creation call; the new process is its **child**. Repeated creation forms a **process tree** rooted at the first process.*

Each process has a unique integer **process identifier** (pid). Figure 7.1 is an illustrative tree: `init` (pid 1) at the root, a login program, an SSH daemon, and `cron` below it, a shell under each login path, and `ls` and `vim` under one shell.

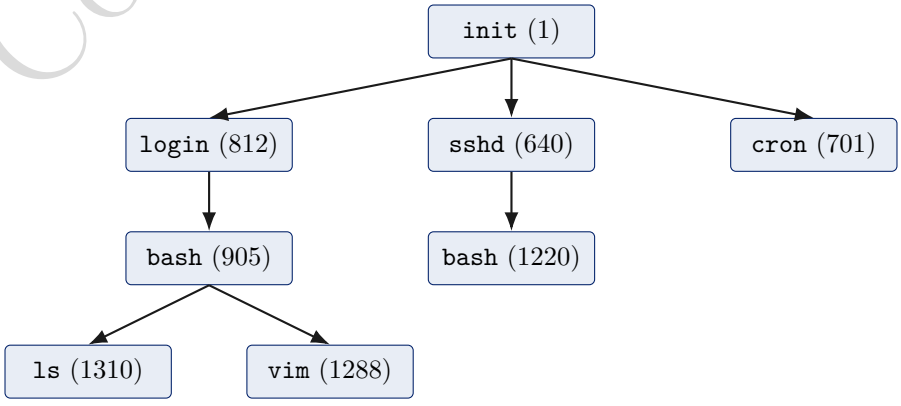


Figure 7.1: A process tree; pids are illustrative — cf. Silberschatz et al., Ch. 3 §3.3, pp. 110–116.

Concepts That Click

How to present it. Follow the coordinate map's levels: draw `init` first, then the three level-1 boxes with their arrows, then the two shells, then `ls` and `vim`. Narrate each level as "these were created by the box above." Write pids last, and say they are illustrative.

The three parental decisions

Creating a child forces three choices: **resources** (the child shares all, a subset, or none of the parent's open files and memory); **execution** (the parent runs concurrently with the child, or waits for some or all children); **address space** (the child duplicates the parent, or has a new program loaded). UNIX answers the third in two steps: `fork` duplicates, `exec` loads.

`fork`: one call, two returns

Definition (`fork`). *`fork()` creates a new process whose address space is a copy of the caller's. Both processes continue at the instruction after the call.*

The child receives `0`; the parent receives the child's pid (positive); on failure the parent receives `-1` and no child exists. The return value is the *only* way the two copies can tell themselves apart. A child learns its own pid with `getpid()` and its parent's with `getppid()`.

Example 7.1 (The canonical program)**.**
```
pid_t pid = fork();
if (pid < 0) {              /* error  */
    perror("fork");
    exit(1);
} else if (pid == 0) {      /* child  */
    execlp("ls", "ls", NULL);
    perror("exec");  /* on failure */
    exit(1);
} else {                    /* parent */
    wait(NULL);
    printf("Child complete\n");
}
```

`fork` returns twice. The child takes the `pid == 0` branch and replaces itself with `ls`; the `perror/exit(1)` run only if `exec` fails. The parent takes the `else` branch and blocks in `wait`. When `ls` finishes, `wait` returns and the parent prints `Child complete`. Same structure as Silberschatz et al., Ch. 3 §3.3.

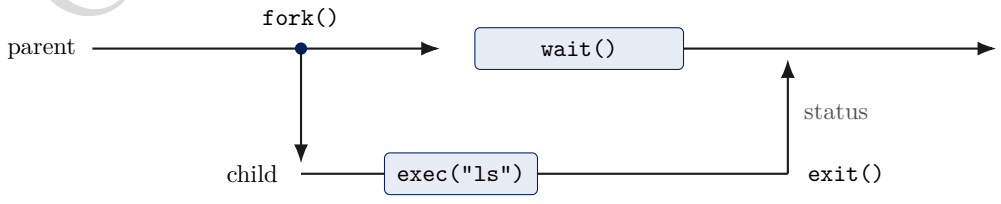


Figure 7.2: The canonical program as a timeline. The parent is blocked in `wait` while the child runs; `exit` wakes it and delivers a status.

How to present Figure 7.2. Draw the parent's horizontal line first and label it. Mark the `fork` dot and drop the vertical arrow to the child's line. Draw `exec("ls")` on the child's line, then `exit()` at its end. Only then draw the `wait` box on the parent's line *above* the child's run, and

Concepts That Click

finally the up-arrow labelled *status*. Saying "the parent is frozen here" while drawing the box is the point of the figure.

Two copies, two variables

After `fork` the address spaces are separate with identical contents; a write in one is invisible to the other. Only open file descriptors are shared (both refer to the same open-file entries).

Board: the lecture's example. *On the board, in order:* (i) write `int x = 10; pid = fork();`; (ii) draw two boxes, "parent" and "child", each with `x = 10`; (iii) in the child box write `x += 5` $\rightarrow$ `15`; (iv) in the parent box write `wait; x -= 5` $\rightarrow$ `5`; (v) write the output `child x=15`, `parent x=5` (verified). *Say:* "Same name, same virtual address, two physical copies."

Board: a second instance. *On the board, in order:* (i) `int x = 5;` child does `x = x * 2`, parent waits then `x = x + 3`; (ii) ask for both printed values before revealing; (iii) write `10` (child) and `8` (parent) — verified. A student who answers 13 is still thinking of one shared `x`; ask them to point to the box where the 10 lives.

> **Teaching note:** Pose the lecture's first Socratic question here, before the answer: "Can the child report its answer in a global variable the parent reads after `wait`?" The answer is no — the global is in the child's copy. The child can return a small integer through `exit`; larger results need IPC (Lectures 14–15).

7.4 Counting Forks

Every `fork` doubles the number of processes executing the next line. Counting is a tree, not a guess.

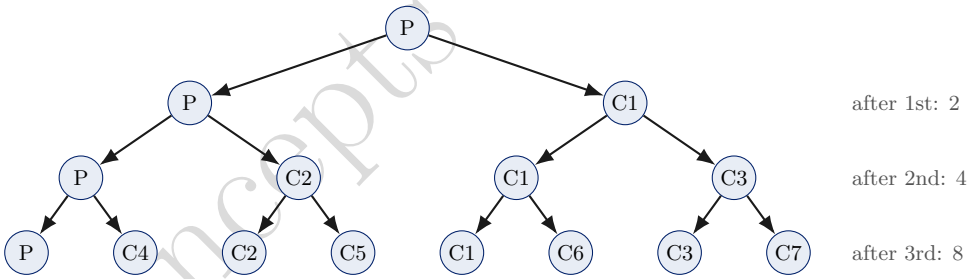


Figure 7.3: Three sequential forks: 2, 4, then 8 processes.

How to present Figure 7.3. Build it one level per `fork`, left to right, and write the running count (2, 4, 8) at the right edge after each level. The convention that matters: a process keeps its name when it forks (the left branch is the same process), and the right branch is the new child. `fork(); fork(); fork(); printf` prints 8 lines; n sequential forks give 2^n processes and $2^n - 1$ new children (verified: 8 for $n = 3$, 16 for $n = 4$).

Board: `printf` inside the loop. *On the board, in order:* (i) write

```
for (i=0;i<3;i++) { fork(); printf("A\n"); }
```

(ii) make a three-row table i / processes after fork / lines; (iii) fill 0/2/2, 1/4/4, 2/8/8; (iv) total 14 (verified). *Prediction trap:* ask for a guess first — many say 8 (they count processes) or 6 (they add $2 \cdot 3$).

Board: a second loop instance. Same table with `i < 4`: $2 + 4 + 8 + 16 = 30$ lines (verified). Ask a student to write the general form, $2^{n+1} - 2$.

Concepts That Click

Short-circuit operators

In `a && b`, `b` runs only if `a` is non-zero; in `a || b`, only if `a` is zero. `fork()` is non-zero in the parent and zero in the child, so the operators choose which copy forks again.

Board: the expression

```
(fork() && fork()) || fork(); printf("y\n");
```

On the board, in order: (i) write P at the top; (ii) first fork: P (non-zero), C1 (zero) — label each edge with the value it saw; (iii) P runs the second fork: P (non-zero), C2 (zero); P's `&&` is true, so P is done — put a tick beside P; (iv) C1's `&&` is false, so it forks: C1, C3; (v) C2's `&&` is false, so it forks: C2, C4; (vi) count P, C1, C2, C3, C4: 5 lines (verified).

Board: `if (fork() && fork()) fork();` (the lecture's Socratic Check 2). C1 sees 0 and skips; P forks C2, which sees 0 and skips; P's condition is true and P forks C3. Four processes, 4 lines (verified). Pose it *before* the solution and let the class vote 3, 4, or 5.

> **Teaching note:** Where students get stuck: they evaluate the whole expression once, for "the program," instead of once *per process*. Insist that every node in the tree carries its own copy of the partially evaluated expression. Labelling each edge with the value `fork` returned on that side (non-zero/0) fixes this almost immediately.

The buffer that gets copied

`printf` writes into a user-space buffer (Lecture 3); `fork` copies the whole address space, including bytes still in that buffer. `printf("hello "); fork(); printf("world\n");` prints `hello world` twice (verified): the unflushed `"hello "` is duplicated and each copy flushes its own buffer. Fix: `fflush(stdout)` before `fork`. Lesson: `fork` copies *state*, not just code.

Board: draw two boxes after the fork, each containing a little buffer with `"hello "` in it, then append `"world\n"` to each. **Board:** second instance — `printf("X"); fork(); fork(); fork(); printf("\n");` with output to a file and no flush gives 8 `X` characters (verified); ask for the count, then for the one-line fix.

7.5 The `exec` Family

Definition (`exec`). *An `exec` call replaces the calling process's memory image — code, data, heap, stack — with a new program loaded from an executable file, and starts it at its entry point.*

The pid and the parent relationship stay the same; open descriptors stay open unless marked close-on-exec. On success `exec` never returns, because the code that called it no longer exists; on failure it returns `-1` and the old program continues (Bovet & Cesati, Ch. 20, pp. 808–834). The name letters: `l` list of arguments, `v` vector, `p` search `PATH`, `e` explicit environment. All are library wrappers; on Linux the one system call underneath is `execve`. Argument lists end with `NULL`; `argv[0]` is the program's name by convention.

Board: code after `exec`. *On the board, in order:* (i) child:

```
execlp("echo","echo","from-exec",NULL); printf("never\n");
```

parent: `wait(NULL); printf("parent\n");`; (ii) ask how many lines; (iii) cross out the `printf("never")` line with a single stroke while saying "this code was in the old image"; (iv) write the output `from-exec`, `parent` — two lines (verified).

Board: `ls > out.txt`. *On the board, in order:* (i) shell `fork`; (ii) child

Concepts That Click

```
fd = open("out.txt", O_WRONLY|O_CREAT|O_TRUNC);
```

(iii) child `dup2(fd, 1)`, `close(fd)`; (iv) child `execlp("ls","ls",NULL)`; (v) shell `wait`. Draw a small descriptor table for the child with slot 1 re-pointed from "terminal" to "out.txt". *Say:* "`ls` writes to descriptor 1 and never knows about the file. This gap between `fork` and `exec` is why UNIX splits creation in two." Redirection, pipes, and changed directories all happen in this gap.

Copying the whole address space only to discard it would be wasteful; modern kernels use **copy-on-write**: pages are shared read-only and copied only on a write, so `fork` costs roughly a copy of the page tables and PCB. Linux also offers `vfork` and `clone` (Bovet & Cesati, Ch. 3, pp. 79–130). Copy-on-write returns in Lecture 26.

Windows uses one step: `CreateProcess` loads a named program into a new process, customization is passed as parameters, and the parent waits with `WaitForSingleObject`. Both designs answer the same three parental decisions; they differ in where the answers are supplied.

> **Teaching note:** Where students get stuck: "`exec` creates a new process." Ask what `getpid()` returns before and after a successful `exec`. The answer — the same number — is the cleanest correction.

7.6 Process Termination

A process terminates after its last statement or by calling `exit(status)`. The kernel releases its memory, open files, and other resources and keeps the small integer status for the parent, which collects it with `wait` (any child; returns the child's pid) or `waitpid` (one named child).

Board: `exit(7)`. Child `exit(7)`; parent `waitpid(pid, &st, 0)`; then `WIFEXITED(st)` is true and `WEXITSTATUS(st)` is 7 (verified). **Board:** second instance — `exit(300)`: only the low 8 bits arrive, so the parent sees $300 - 256 = 44$ (verified). Use it to explain why the status is "a small integer."

A parent may also end a child (`kill` on UNIX, `TerminateProcess` on Windows) — only a parent or a suitably privileged user may — because the child exceeded its resources, its task is no longer required, or the parent is exiting on a system that does not let children outlive parents. On such systems the OS performs **cascading termination**: ending a parent ends all its descendants (Silberschatz et al., Ch. 3 §3.3).

Definition (Zombie). *A process that has terminated but whose parent has not yet called `wait`; its resources are freed, but its PCB entry (pid, exit status) remains.*

Definition (Orphan). *A process whose parent terminated without waiting for it; UNIX re-parents it to `init` (pid 1), which calls `wait` periodically.*

Every child is briefly a zombie; the problem is a parent that never waits. Many zombies exhaust the pid table even though they use no memory. (Standard UNIX treatment; Bovet & Cesati, Ch. 3, pp. 79–130.)

How to present Figure 7.4. Draw the four solid boxes left to right with their arrow labels (`fork`, `exit`, parent `wait`) *above* each arrow. Only after the solid path is understood, draw the orphan box below and the two dashed legs. Say clearly that "zombie" is a bookkeeping state, not a running program.

Board: timeline of a zombie. Child exits at $t = 2\,\text{s}$, parent waits at $t = 10\,\text{s}$: mark "running" on $[0, 2)$, "zombie" on $[2, 10)$, "reaped" after 10. **Board:** timeline of an orphan. Parent exits at $t = 1\,\text{s}$, child exits at $t = 2\,\text{s}$: "running, parent = P" on $[0, 1)$, "orphan, parent = `init`" on $[1, 2)$, then briefly a zombie until `init` reaps it.

Concepts That Click

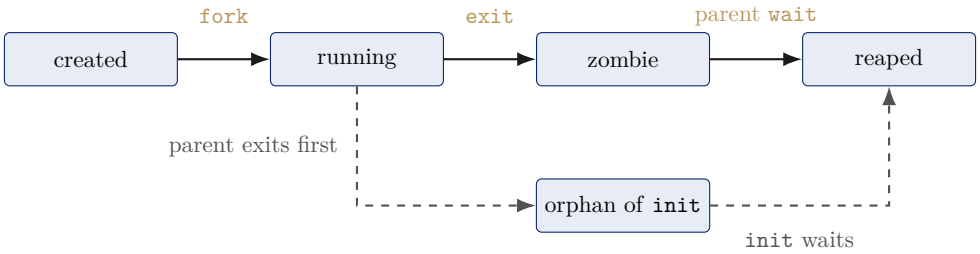


Figure 7.4: The life cycle. Solid: the normal path. Dashed: the parent exits first.

Live hook (Socratic Check 3): "A server forks one child per request and never waits; children finish in milliseconds. What piles up?" Zombies: pids and statuses stay in the process table; memory is released but pids run out. Fix: reap with `wait`/`waitpid`, for example in a loop with `WNOHANG`.

The shell loop

Read a line (e.g. `ls -l`); `pid = fork()`; in the child set up redirection and `execvp("ls", argv)`, and if `exec` returns print an error and `exit(127)`; in the parent `waitpid(pid, &status, 0)` unless the line ended with `&`, in which case return to the prompt immediately; repeat. Per foreground command: `fork` (shell), `execvp` (child), `exit` (child), `wait` (shell) — four process-control calls, the category from Lecture 3. Close the lecture by pointing back at the four verbs from the opening board.

7.7 Summary

- Processes form a tree; every process but the first has a parent.
- `fork` returns twice: `0` to the child, the child's pid to the parent, `-1` on failure.
- n sequential forks give 2^n processes ($2^n - 1$ children); loops and short-circuit operators change the count — draw the tree, one evaluation per process.
- `fork` copies state, including unflushed stdio buffers.
- `exec` keeps the pid and replaces the program; it returns only on failure.
- `exit` delivers a status (low 8 bits); `wait`/`waitpid` collect it. Unreaped child = zombie; parentless child = orphan adopted by `init`.
- Misconceptions to pre-empt: the child gets the child's pid; parent and child share variables; `exec` creates a process; code after a successful `exec` runs; a zombie uses CPU and memory.
- Next lecture: CPU scheduling — criteria (TAT, WT, RT), FCFS and SJF (Silberschatz Ch. 5 §5.1–5.3; Stallings Ch. 9 §9.1–9.2).

Concepts That Click